Lecture 10: Non- and semi-parametric methods for prediction and estimation; high-dimensional methods

ECON 7510
Cornell University
Adam Harris

Motivation

In the course of estimating an economic model, we may need to:

- estimate the transition process for a state variable x_t, i.e. estimate the distribution of $x_{t+1} \mid x_t, a_t$.
- estimate the mapping from some state variable to some outcome, e.g., patient's probability of heart attack as a function of observables.
- classify observations into different groups.
- reduce the dimensionality of some high-dimensional state variable.

In many cases, we may want to avoid making strong functional-form assumptions and may want a more flexible approach.

Intro to non- and semi-parametric methods

Non- and semi-parametric methods

A motivating example: See `Lec10.jl`

Evaluating prediction methods

- **Question**: What value of K should we pick?

Evaluating prediction methods

- **Question**: What value of K should we pick?
  - $K = 1$ will always give smallest MSE. **Question**: What is MSE for $K = 1$?

Evaluating prediction methods

- **Question**: What value of K should we pick?
  - $K = 1$ will always give smallest MSE. **Question**: What is MSE for $K = 1$?
  - **Question**: Does that mean $K = 1$ is the best choice?

Evaluating prediction methods

- **Question**: What value of K should we pick?
  - $K = 1$ will always give smallest MSE. **Question**: What is MSE for $K = 1$?
  - **Question**: Does that mean $K = 1$ is the best choice?
- Stepping back, our goal is getting a generally useful prediction, not merely memorizing the training data. (Memorizing the training data is called **overfitting**.)
- To that end, we make a distinction between training and test performance.

See `Lec10.jl`

Evaluating prediction methods

More generally, with non-parametric methods, we face a tradeoff between **bias** and **variance**.

$$\text{MSE}\left(\hat{y}\right) = \text{Bias}\left(\hat{y}\right)^2 + \text{Var}\left(\hat{y}\right)$$

Might be willing sacrifice some information about local behavior to avoid capturing noise.

Evaluating prediction methods

- In the example above, we drew an additional sample from the DGP to evaluate OOS performance.
  Question: Is that a good approach in general?

Evaluating prediction methods

- In the example above, we drew an additional sample from the DGP to evaluate OOS performance.
  Question: Is that a good approach in general?

- Of course, we don't generally know the DGP, so we can't just draw an additional sample.
  Question: What can we do instead?

Evaluating prediction methods

- In the example above, we drew an additional sample from the DGP to evaluate OOS performance.
  Question: Is that a good approach in general?

- Of course, we don't generally know the DGP, so we can't just draw an additional sample.
  Question: What can we do instead?

  - Split our sample into train and test and select K by evaluating test-sample performance.

Evaluating prediction methods

- In the example above, we drew an additional sample from the DGP to evaluate OOS performance.
  Question: Is that a good approach in general?

- Of course, we don't generally know the DGP, so we can't just draw an additional sample.
  Question: What can we do instead?

  - Split our sample into train and test and select K by evaluating test-sample performance.
  - Better yet, do this splitting many times and select K that does best on average across held-out samples (**cross-validation**).

Evaluating prediction methods

- In the example above, we drew an additional sample from the DGP to evaluate OOS performance.
  Question: Is that a good approach in general?

- Of course, we don't generally know the DGP, so we can't just draw an additional sample.
  Question: What can we do instead?

  - Split our sample into train and test and select K by evaluating test-sample performance.
  - Better yet, do this splitting many times and select K that does best on average across held-out samples (**cross-validation**).
  - **Regularization**: Impose an explicit penalty on more complex models.

Evaluating prediction methods

- In the example above, we drew an additional sample from the DGP to evaluate OOS performance.
  Question: Is that a good approach in general?

- Of course, we don't generally know the DGP, so we can't just draw an additional sample.
  Question: What can we do instead?

  - Split our sample into train and test and select K by evaluating test-sample performance.
  - Better yet, do this splitting many times and select K that does best on average across held-out samples (**cross-validation**).
  - **Regularization**: Impose an explicit penalty on more complex models.
    Question: Can you think of commonly-used methods that do this?

Evaluating prediction methods

- In the example above, we drew an additional sample from the DGP to evaluate OOS performance.
  Question: Is that a good approach in general?

- Of course, we don't generally know the DGP, so we can't just draw an additional sample.
  Question: What can we do instead?

  - Split our sample into train and test and select K by evaluating test-sample performance.
  - Better yet, do this splitting many times and select K that does best on average across held-out samples (**cross-validation**).
  - **Regularization**: Impose an explicit penalty on more complex models.
    Question: Can you think of commonly-used methods that do this?

    » LASSO, Ridge are like OLS regressions with penalty based on magnitude of β coefficients.

    » Bayesian Information Criterion for model selection:

$$\text{BIC}(M) = (\#\text{ of parameters in } M) \log N - 2 \log \text{Likelihood}(M)$$

Semi-parametric methods for estimating distributions

Motivation

Suppose you want to estimate the AR(1) transition process

$$x_t = x_{t-1} + \eta_t$$

where $\eta_t \sim$ iid F is a random variable.

- Observe many realizations of x_t (and therefore η_t).
- Want to estimate the distribution F.
  Question: How do we proceed?

Motivation

Suppose you want to estimate the AR(1) transition process

$$x_t = x_{t-1} + \eta_t$$

where $\eta_t \sim$ iid F is a random variable.

- Observe many realizations of x_t (and therefore η_t).
- Want to estimate the distribution F.
  Question: How do we proceed?
  - Could assume functional form for F.

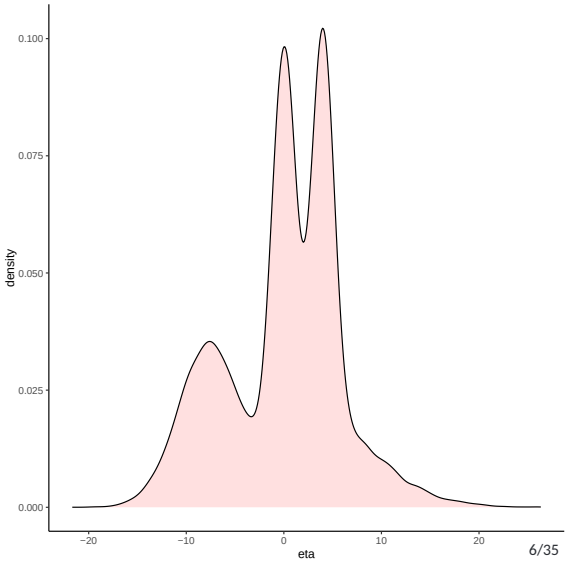
Motivation

Suppose you want to estimate the AR(1) transition process

$$x_t = x_{t-1} + \eta_t$$

where $\eta_t \sim$ iid F is a random variable.

- Observe many realizations of x_t (and therefore η_t).
- Want to estimate the distribution F.
  Question: How do we proceed?
  - Could assume functional form for F.
  - Suppose you look at a histogram of η and it doesn't look like any distribution you recognize (e.g., multiple modes).



Motivation

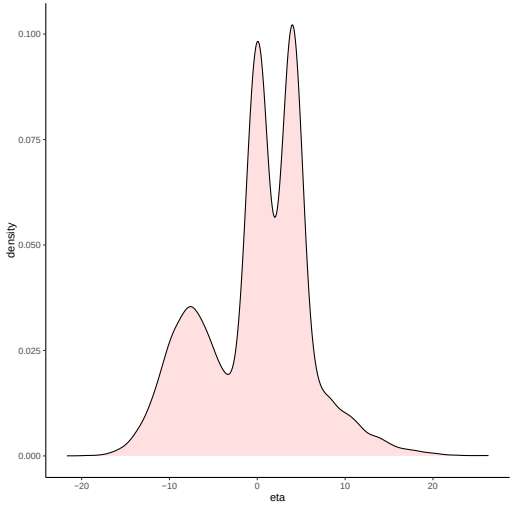
May be well approximated by a *mixture* of distributions you do recognize.

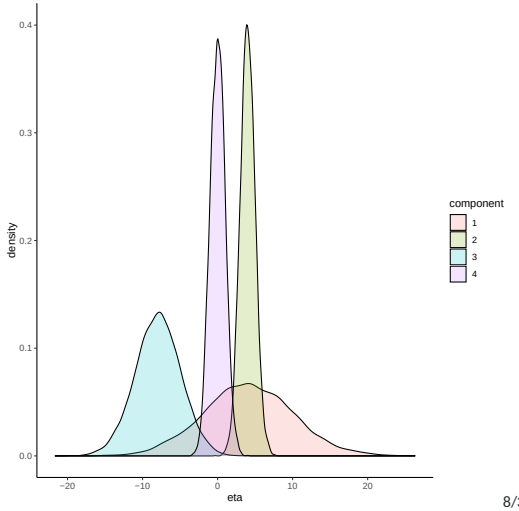
$$\eta \sim \begin{cases} \Phi(\cdot \mid \theta_1) & \text{w.p. } \pi_1 \\ \Phi(\cdot \mid \theta_2) & \text{w.p. } \pi_2 \\ \vdots & \\ \Phi(\cdot \mid \theta_K) & \text{w.p. } \pi_K \end{cases}$$

with $\sum_k^K \pi_k = 1$.

Motivation

May be well approximated by a *mixture* of distributions you do recognize.





Universal approximation

Proposition: For any continuous distribution F, you can find a value of K, weights $\{\pi_1, \ldots, \pi_K\}$, and parameters $\{\theta_1, \ldots, \theta_K\}$ such that the Gaussian mixture is an arbitrarily good approximation of the distribution F.

Identification

Suppose you know that data were generated from a Gaussian Mixture model. Is the model identified?

- **Question**: Suppose number of components K is known. Are the components identified?

Identification

Suppose you know that data were generated from a Gaussian Mixture model. Is the model identified?

- **Question**: Suppose number of components K is known. Are the components identified? Yes, though of course the components' labels $k = 1, \ldots, K$ are arbitrary.

Identification

Suppose you know that data were generated from a Gaussian Mixture model. Is the model identified?

- **Question**: Suppose number of components K is known. Are the components identified? Yes, though of course the components' labels $k = 1, \ldots, K$ are arbitrary.

- **Question**: Suppose number of components K is unknown. Are K and the components identified?

Identification

Suppose you know that data were generated from a Gaussian Mixture model. Is the model identified?

- **Question**: Suppose number of components K is known. Are the components identified? Yes, though of course the components' labels $k = 1, \ldots, K$ are arbitrary.

- **Question**: Suppose number of components K is unknown. Are K and the components identified? Yes, if we impose "unique components" assumption.

Estimation of finite mixture models

Given the number of components K, the log-likelihood is

$$\ell(\theta_1, \ldots, \theta_K) = \sum_i \log \sum_k \pi_k f(x_i \mid \theta_k)$$

Estimation of finite mixture models

Given the number of components K, the log-likelihood is

$$\ell(\theta_1, \ldots, \theta_K) = \sum_i \log \sum_k \pi_k f(x_i \mid \theta_k)$$

— **Question**: Think about solving this MLE problem using standard optimization techniques. What issues might arise?

Estimation of finite mixture models

Given the number of components K, the log-likelihood is

$$\ell(\theta_1, \ldots, \theta_K) = \sum_i \log \sum_k \pi_k f(x_i \mid \theta_k)$$

- **Question**: Think about solving this MLE problem using standard optimization techniques. What issues might arise? Generally non-concave. Potentially many local maxima.
- **Question**: How do we generally overcome the local optima issue?

Estimation of finite mixture models

Given the number of components K, the log-likelihood is

$$\ell(\theta_1, \ldots, \theta_K) = \sum_i \log \sum_k \pi_k f(x_i \mid \theta_k)$$

- **Question**: Think about solving this MLE problem using standard optimization techniques. What issues might arise? Generally non-concave. Potentially many local maxima.
- **Question**: How do we generally overcome the local optima issue? Multi-start optimization and/or stochastic methods. But these methods are slow.

Estimation of finite mixture models

Given the number of components K, the log-likelihood is

$$\ell(\theta_1, \ldots, \theta_K) = \sum_i \log \sum_k \pi_k f(x_i \mid \theta_k)$$

- **Question**: Think about solving this MLE problem using standard optimization techniques. What issues might arise? Generally non-concave. Potentially many local maxima.
- **Question**: How do we generally overcome the local optima issue? Multi-start optimization and/or stochastic methods. But these methods are slow.
- Is there a faster way to solve this particular optimization problem that we can perform many times? Luckily, yes.
  - → EM algorithm exploits the structure of this MLE problem to get to an optimum more quickly.
  - → No guarantee of finding the global optimum, but its speed makes massive multi-start feasible.

EM algorithm

EM algorithm is an example of a *block coordinate descent* algorithm. Suppose we have the optimization problem

$$\min_{\beta} f(\beta)$$

- Partition $\beta = (\beta_1, \beta_2)$.
- Iterate between two steps:
  - Use gradient to update β_1 while holding β_2 fixed.
  - Use gradient to update β_2 while holding β_1 fixed.
- **Question**: Why might this approach be useful?

EM algorithm

EM algorithm is an example of a *block coordinate descent* algorithm. Suppose we have the optimization problem

$$\min_{\beta} f(\beta)$$

- Partition $\beta = (\beta_1, \beta_2)$.
- Iterate between two steps:
  - Use gradient to update β_1 while holding β_2 fixed.
  - Use gradient to update β_2 while holding β_1 fixed.
- **Question**: Why might this approach be useful? No reason why this is a useful approach in general, but in some cases, we can exploit the structure of the problem and choose partition in a way that makes separate update steps faster/easier than updating all of the parameters jointly.

EM algorithm

- EM = Expectation-Maximization.
- It's an iterative optimization algorithm.
- Commonly used to find maximum likelihood estimates when the model depends on unobserved latent variables.
- Examples include:
  - Missing data problems.
  - Mixture models.
  - Hidden Markov Models (HMMs).

The EM Algorithm - Overview

- The algorithm has two main steps:
  1. **E-step (Expectation step)**: Calculate the expected value of the log-likelihood function, with respect to the current estimate of the distribution for the latent variables.
  2. **M-step (Maximization step)**: Find the parameter that maximizes the expected log-likelihood found on the E-step.
- Iterate these two steps until convergence.

What is the EM Algorithm?

Application to Gaussian Mixture Models

- In the context of GMMs, the latent variables indicate the Gaussian component each data point belongs to.
- EM iteratively improves parameter estimates by alternating between an Expectation (E) step and a Maximization (M) step.

What is the EM Algorithm?

Application to Gaussian Mixture Models

- In the context of GMMs, the latent variables indicate the Gaussian component each data point belongs to.
- EM iteratively improves parameter estimates by alternating between an Expectation (E) step and a Maximization (M) step.

1. **E-step**: Given the components' parameters (means, variances, and weights), compute the probability that each data point belongs to each Gaussian component ("responsibilities").
2. **M-step**: Given these "responsibilities," update the parameters (means, variances, and weights) of the Gaussian components..
3. Repeat until convergence.

E-step: Calculating Responsibilities

- Take $\{\mu_k, \sigma_k, \pi_k\}_{k=1}^{K}$ as given.
- Calculate the posterior probability (responsibility) $\gamma_{i,k}$ that data point x_i was generated by Gaussian component k.

$$\gamma_{i,k} = \Pr(i \text{ was generated by component } k \mid x_i)$$ (Apply Bayes' Rule?)

E-step: Calculating Responsibilities

- Take $\{\mu_k, \sigma_k, \pi_k\}_{k=1}^K$ as given.
- Calculate the posterior probability (responsibility) $\gamma_{i,k}$ that data point x_i was generated by Gaussian component k.

$$
\begin{aligned}
\gamma_{i,k} &= \Pr\left(i \text{ was generated by component } k \mid x_i\right) \\
&= \frac{\Pr\left(x_i \mid i \text{ was generated by component } k\right) \Pr\left(i \text{ was generated by component } k\right)}{\Pr\left(x_i\right)} \\
&= \frac{\pi_k \phi(x_i \mid \mu_k, \sigma_k)}{\sum_{j=1}^K \pi_j \phi(x_i \mid \mu_j, \sigma_j)}
\end{aligned}
$$

- Where $\phi(x_i \mid \mu_k, \sigma_k)$ is the Gaussian probability density function.

M-step: Updating Parameters

Holding the weights π fixed, we have closed form for MLE estimators of

- Update the parameters using the responsibilities calculated in the E-step.

$$\mu_k^{(t+1)} = \frac{\sum_{i=1}^{N} \gamma_{i,k} x_i}{\sum_{i=1}^{N} \gamma_{i,k}}$$

$$\sigma_k^{(t+1)} = \frac{\sum_{i=1}^{N} \gamma_{i,k} (x_i - \mu_k^{(t+1)})^2}{\sum_{i=1}^{N} \gamma_{i,k}}$$

$$\pi_k^{(t+1)} = \frac{1}{N} \sum_{i=1}^{N} \gamma_{i,k}$$

- N is the total number of data points.

EM algorithm

- The EM algorithm converges when the change in the log-likelihood function is below a set threshold, or after a set number of iterations.
- Log-likelihood for GMM:

$$L(\theta) = \sum_{i=1}^{N} \log \left(\sum_{k=1}^{K} \pi_k \phi(x_i \mid \mu_k, \sigma_k) \right) \tag{2.1}$$

- **Question**: We have to specify the value of K. What value should we choose?

EM algorithm

- The EM algorithm converges when the change in the log-likelihood function is below a set threshold, or after a set number of iterations.
- Log-likelihood for GMM:

$$L(\theta) = \sum_{i=1}^{N} \log \left(\sum_{k=1}^{K} \pi_k \phi(x_i \mid \mu_k, \sigma_k) \right) \tag{2.1}$$

- **Question**: We have to specify the value of K. What value should we choose?
  - Most common approach: Carry out EM algorithm for various values of K and evaluate BIC for each.

See `Lec10-EM.jl`

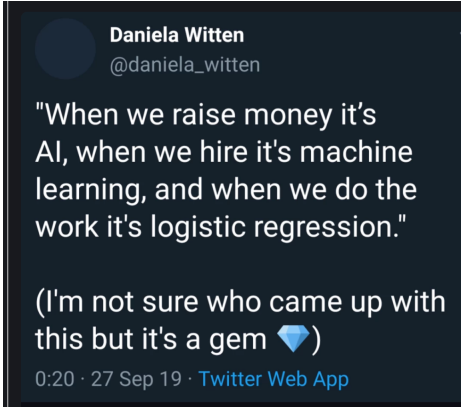
ML methods for prediction, function approximation

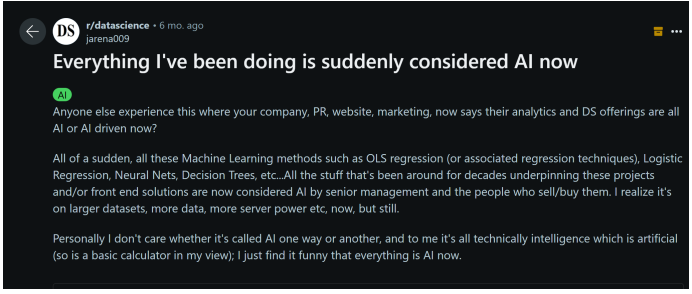
What is machine learning?

Wikipedia: “a field of study in artificial intelligence concerned with the development and study of statistical algorithms that can learn from data and generalize to unseen data”

What is machine learning?

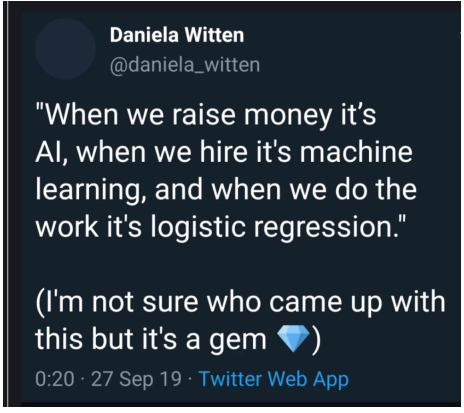
Wikipedia: “a field of study in artificial intelligence concerned with the development and study of statistical algorithms that can learn from data and generalize to unseen data”

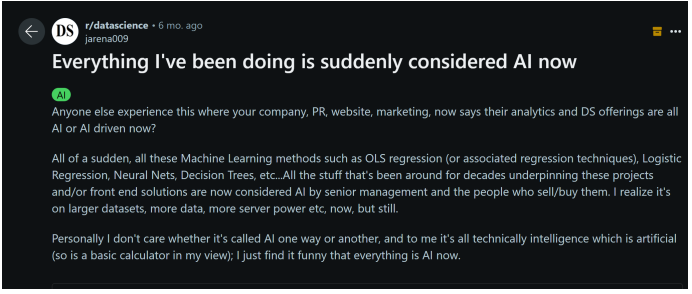




What is machine learning?

Wikipedia: “a field of study in artificial intelligence concerned with the development and study of statistical algorithms that can learn from data and generalize to unseen data”





Distinctions that are still somewhat useful: (1) Non- or semi-parametric and (2) works well with high-dimensional data.

General setup

- You have a data sample of predictors X (high-dimensional), outcomes y, which could be discrete ("classification") or continuous ("regression").

- Want to learn mapping $y = f(x)$, but don't necessarily need f to be interpretable.

General setup

- You have a data sample of predictors X (high-dimensional), outcomes y, which could be discrete ("classification") or continuous ("regression").
- Want to learn mapping $y = f(x)$, but don't necessarily need f to be interpretable.
- **Methods**:
  - KNN model (flexible, but performs poorly when X is high-dimensional)
  - LASSO, Ridge, generalized linear models (most parametric)
  - Tree-based models (more flexible)
  - Neural net models (most flexible)

Tree-based methods

Introduction to GBDT

- Gradient Boosted Decision Trees (GBDT) is an ensemble learning method.
- Combines the predictions of multiple decision trees.
- Utilizes gradient boosting, an iterative technique.

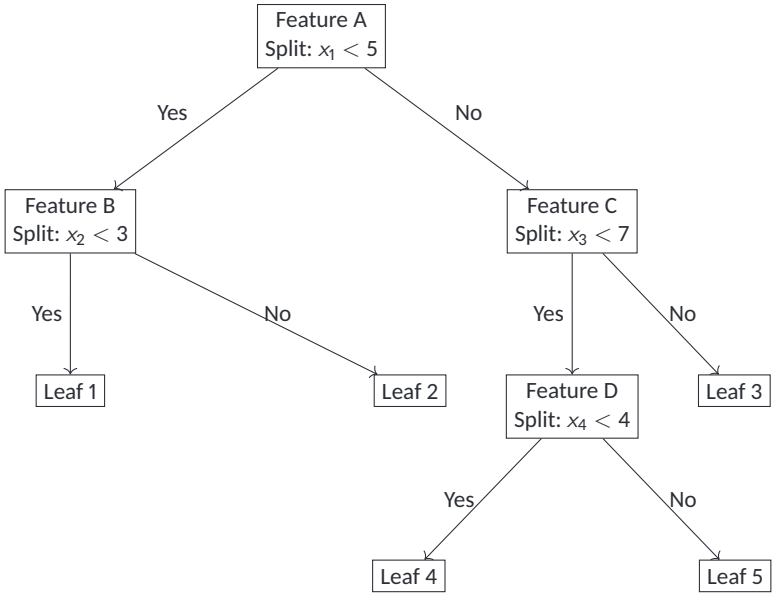
Introduction to GBDT

- Gradient Boosted Decision Trees (GBDT) is an ensemble learning method.
- Combines the predictions of multiple decision trees.
- Utilizes gradient boosting, an iterative technique.

Decision trees:

- Decision Trees are simple, non-parametric models.
- Used for classification and regression tasks.
- Split data into subsets based on feature values.

Decision Trees



Decision Trees

- On their own, decisions trees are *weak learners*, i.e., its predictions aren't much better than a random guess.

Decision Trees

- On their own, decisions trees are *weak learners*, i.e., its predictions aren't much better than a random guess.

- Can nevertheless be useful if we combine many trees together (*ensemble method*)

  1. Bagging—random forest
  2. Boosting—GBDT

Random forest

- Draw a random subset and train a tree.
- Repeat many times and then average the trees' predictions.

$$F(x) = \frac{1}{M} \sum_{m=1}^{M} F_m(x)$$

Gradient boosted decision trees

- Boosting is another method to improve predictive accuracy.
- Trains models sequentially, each new model correcting errors of the previous one.
- Uses gradient descent to minimize loss function.
- But it turns out that for MSE loss, gradient is equal to the residual.

GBDT Algorithm

1. Initialize with a constant model, $F_0(x) = \arg\min_\gamma \sum_{i=1}^{N} L(y_i, \gamma)$.

2. For each iteration $m = 1, 2, \ldots, M$:

   - Compute residuals: $r_{im} = y - F_m(x_i)$.
   - Fit a decision tree to predict r_m. Call this tree h_m.
   - Update model: $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$.

Regularizing GBDTs

- Slow learning + early stopping: Choose small η and smallish M.
- Put fixed limit on tree depth.
- Only continue building a tree if improvement in loss exceeds some threshold.
- Put L1 or L2 regularization penalty on weights—discourages overly strong reliance on any one feature.

Implementation of GBDT

See `Lec10-GBDT.jl`

Discussion of GBDT

— **Question**: Can we interpret trained GBDT model?

Discussion of GBDT

- **Question**: Can we interpret trained GBDT model?
- **Question**: What is the derivative of $F_M(x)$ with respect to some particular x?

Discussion of GBDT

- **Question**: Can we interpret trained GBDT model?
- **Question**: What is the derivative of $F_M(x)$ with respect to some particular x?
- One big upside of GBDTs: They can handle missing values.

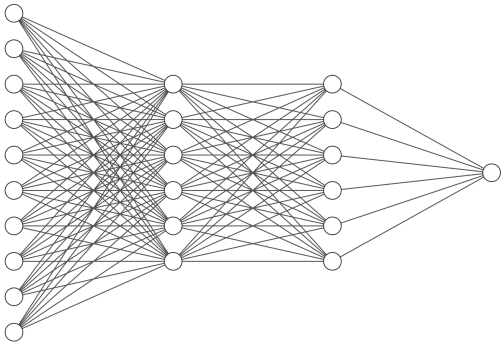
Neural networks

Brief overview of Neural Networks

- **Prediction problem**: $\hat{y}_i = f(x_i; \theta)$ where $x_i \in \mathbb{R}^N$
- **Training**: Find parameter values θ that minimize some objective, e.g.,
  - SSE: $\sum_i (\hat{y}_i - y_i)^2$
  - Log-loss (for binary outcome): $\sum_i [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$

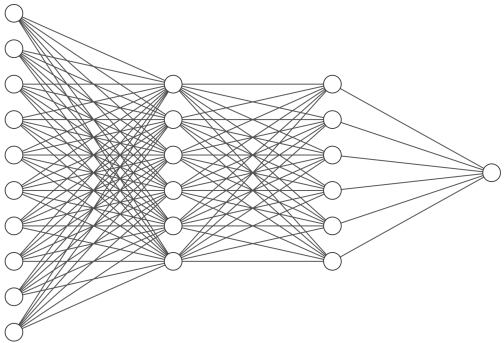
Anatomy of a feed-forward neural network

- Parameters: Weights $\mathbf{W}$ and biases $\mathbf{b}$
- $\mathbf{a}^{(l)} = \quad \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$



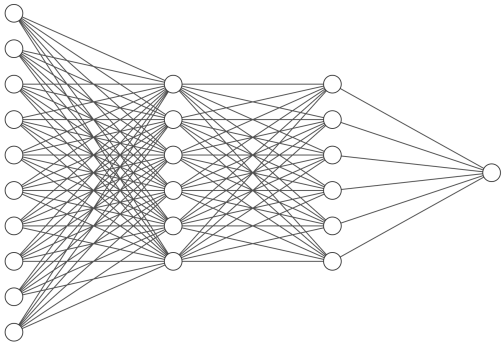
Anatomy of a feed-forward neural network

- Parameters: Weights $\mathbf{W}$ and biases $\mathbf{b}$
- $\mathbf{a}^{(l)} = \quad \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$
- **Question**: As written, what kind of function is this?



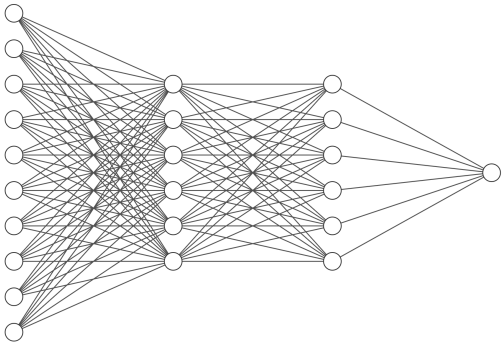
Anatomy of a feed-forward neural network

- Parameters: Weights **W** and biases **b**
- $\mathbf{a}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$
- **Question**: As written, what kind of function is this?
- Introducing non-linear σ makes the NN a very flexible function.



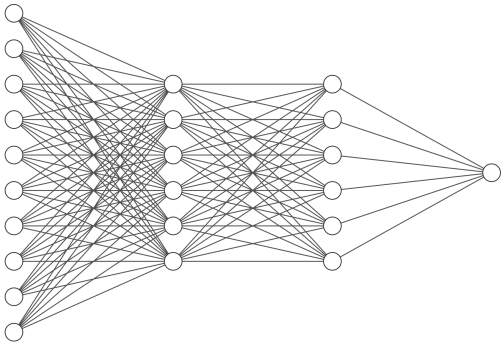
Anatomy of a feed-forward neural network

- Parameters: Weights $\mathbf{W}$ and biases $\mathbf{b}$
- $\mathbf{a}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$
- **Question**: As written, what kind of function is this?
- Introducing non-linear σ makes the NN a very flexible function.
- **Universal function approximator theorem**: If network is sufficiently *deep* or sufficiently *wide*, it can approximate any* function with arbitrary degree of accuracy.



Anatomy of a feed-forward neural network

- Parameters: Weights $\mathbf{W}$ and biases $\mathbf{b}$
- $\mathbf{a}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$
- **Question**: As written, what kind of function is this?
- Introducing non-linear σ makes the NN a very flexible function.
- **Universal function approximator theorem**: If network is sufficiently *deep* or sufficiently *wide*, it can approximate any* function with arbitrary degree of accuracy.



- **Comprehension check**: Suppose a NN has zero hidden layers. What is another name for this model?

Why is it feasible to train neural networks?

— At first glance, the task of finding good parameter values θ seems quite difficult. Number of parameters:

$$(J+1)K + (HK+1)(K+1)$$

where $J = \dim(x)$, H is number of hidden layers, and K is width of each hidden layer.

Why is it feasible to train neural networks?

- At first glance, the task of finding good parameter values θ seems quite difficult. Number of parameters:

$$(J+1)K + (HK+1)(K+1)$$

  where $J = \dim(x)$, H is number of hidden layers, and K is width of each hidden layer.

- But we can compute the gradient analytically using a trick that exploits the structure of the network: *Back-propagation*.

Back-propagation

- Parameters: Weights $\mathbf{W}$ and biases $\mathbf{b}$
- $\mathbf{a}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$
- Note that it's easy to compute
  - $\frac{\partial \hat{y}}{\partial \mathbf{a}^{(3)}}$
  - $\frac{\partial \hat{y}}{\partial \mathbf{W}^{(3)}}$ and $\frac{\partial \hat{y}}{\partial \mathbf{b}^{(3)}}$

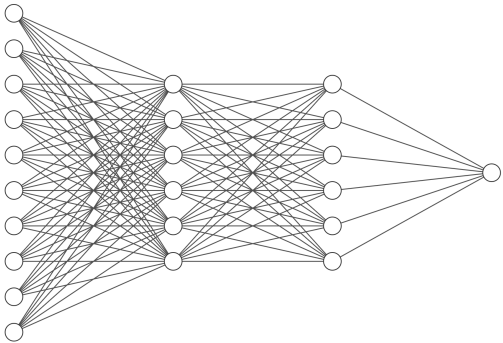
Back-propagation

- Parameters: Weights $\mathbf{W}$ and biases $\mathbf{b}$
- $\mathbf{a}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$
- Note that it's easy to compute
  - $\frac{\partial \hat{y}}{\partial \mathbf{a}^{(3)}}$
  - $\frac{\partial \hat{y}}{\partial \mathbf{W}^{(3)}}$ and $\frac{\partial \hat{y}}{\partial \mathbf{b}^{(3)}}$
- What about parameters of earlier layers?

Back-propagation

- Parameters: Weights $\mathbf{W}$ and biases $\mathbf{b}$
- $\mathbf{a}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$
- Note that it's easy to compute
  - $\frac{\partial \hat{y}}{\partial \mathbf{a}^{(3)}}$
  - $\frac{\partial \hat{y}}{\partial \mathbf{W}^{(3)}}$ and $\frac{\partial \hat{y}}{\partial \mathbf{b}^{(3)}}$
- What about parameters of earlier layers? By the Chain Rule,

$$\frac{\partial \hat{y}}{\partial \mathbf{W}^{(2)}} = \frac{\partial \hat{y}}{\partial \mathbf{a}^{(3)}} \frac{\partial \mathbf{a}^{(3)}}{\partial \mathbf{W}^{(2)}}$$
$$\frac{\partial \hat{y}}{\partial \mathbf{b}^{(2)}} = \frac{\partial \hat{y}}{\partial \mathbf{a}^{(3)}} \frac{\partial \mathbf{a}^{(3)}}{\partial \mathbf{b}^{(2)}}$$



Reflection on neural networks

Training:

- We can compute the gradient of loss function using back-propagation.
- **Question**: Does that mean training is easy?

Reflection on neural networks

Training:

- We can compute the gradient of loss function using back-propagation.
- **Question**: Does that mean training is easy? No.
  - Highly non-convex optimization problem.
  - Regularization is critical, and can be a bit trickier.

Reflection on neural networks

Training:

- We can compute the gradient of loss function using back-propagation.
- **Question**: Does that mean training is easy? No.
  - Highly non-convex optimization problem.
  - Regularization is critical, and can be a bit trickier.

Broad thoughts:

- NNs are extremely flexible, continuous, and continuously differentiable.
- But they're not as easy to train out-of-the-box as GBDTs.

Next time

Starting demand estimation